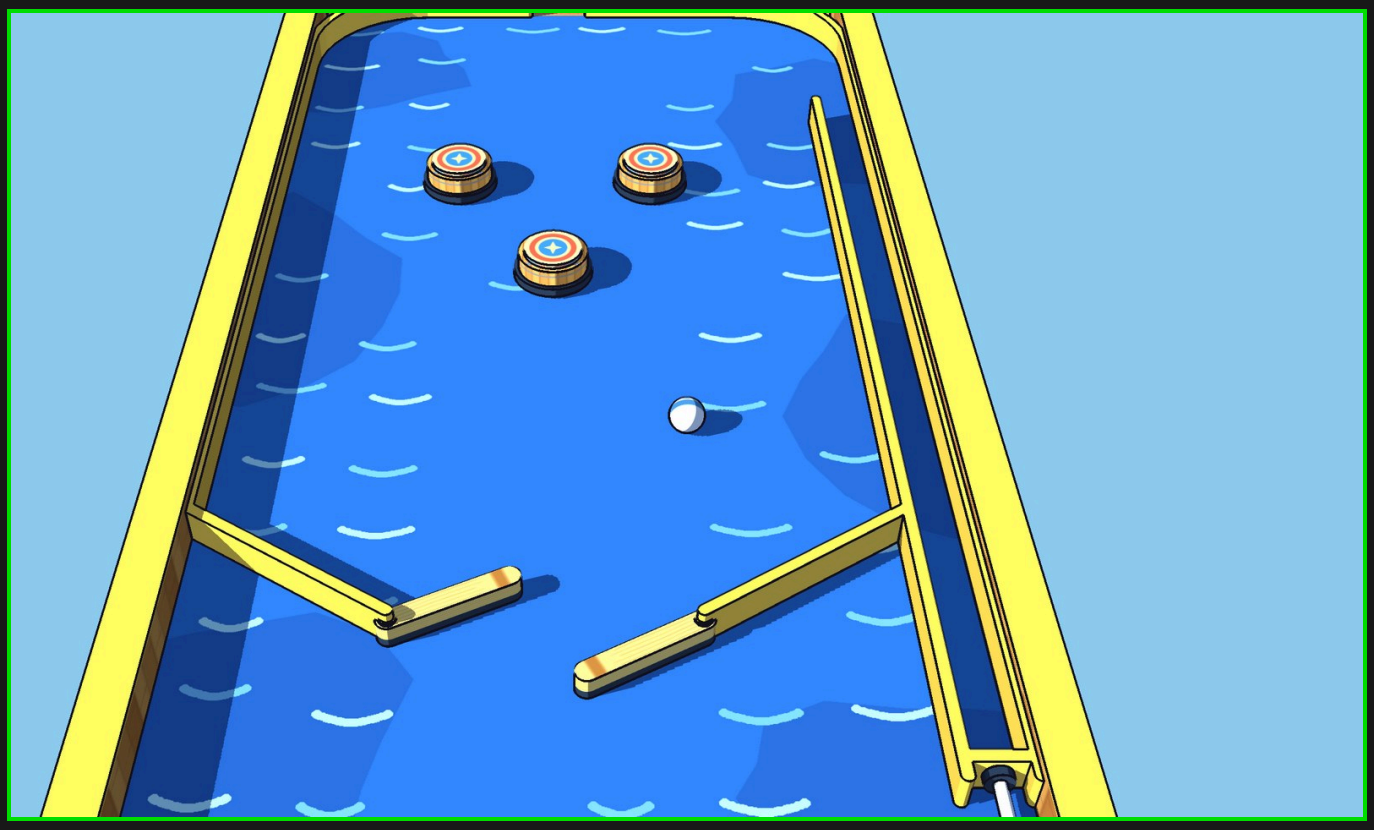
Final Project: Toon Pinball (Cel Shading)

Pinball Source (hw_final_toon)

Play it on GitHub Pages

For our final project we built a pinball game in WebGL 2 and gave it a cartoon look, inspired by the cel-shaded style of "The Legend of Zelda: The Wind Waker". This write-up explains the code behind that look: how `render.js` turns the Blinn-Phong lighting from class into cel shading, where the ink outlines come from, and why each number in the shaders is what it is. All code below is copied from `hw_final_toon/render.js`.



Flippers `Z` `←` `/` `→` Launch `Space` Camera `C` Tilt `5` `6` Zoom `scroll`

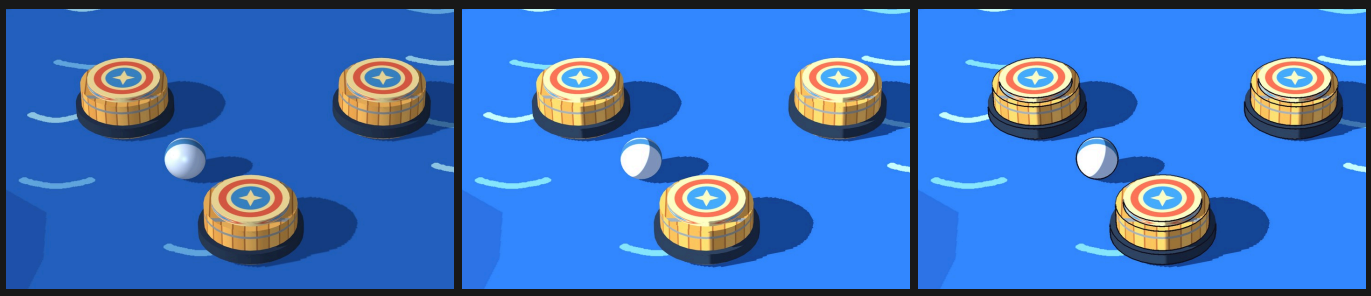
Things to look for (in the game or in the picture above):

- Each bumper and the ball have exactly two tones, a lit one and a shadow one, with a sharp edge between them.
- Highlights are solid dots, not soft glows.
- Dark lines trace silhouettes (where an object ends against what is behind it) and creases (the rings on the bumpers, the tops of the walls).
- Hold `5` or `6` to tilt the board: the bands slide as surfaces turn toward or away from the light.
- Scroll to zoom in: the lines stay the same thickness, because they are drawn in screen space.

How does the cel shading work?

Cel shading imitates hand-painted animation cels: flat areas of color, a hard line between light and shadow, a solid highlight, and dark ink lines around shapes. We get there with four changes to the lighting pipeline we built in class:

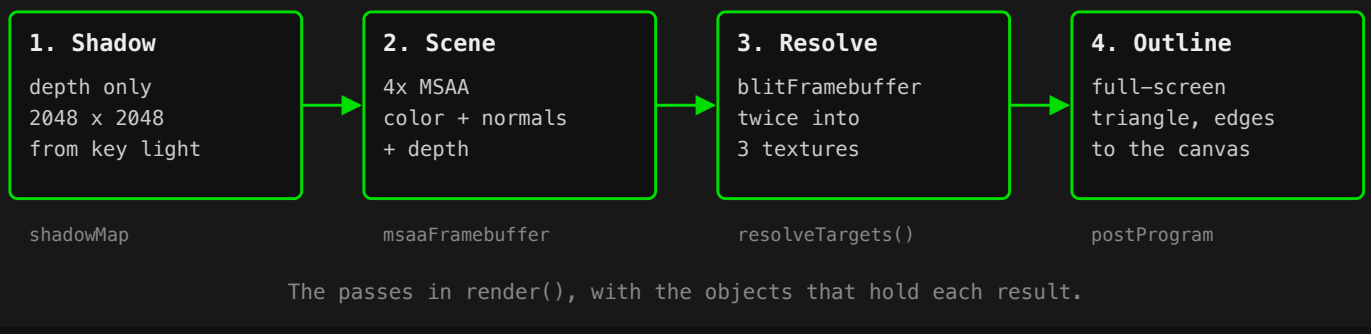
- **Two-tone light.** Lambert's `N·L` goes through a `smoothstep`, so a surface is lit or not.
- **A highlight that is a dot.** The Blinn-Phong specular term goes through a `smoothstep` too.
- **Hard shadows.** The shadow-map test is sharpened the same way.
- **Ink outlines.** A post-process pass finds edges in the depth and normal buffers and paints them dark.



The same frame three ways. Left: smooth Blinn-Phong, as in class. Middle: our cel lighting, specular and shadows, without outlines. Right: the final image with the outline pass.

One frame, four passes

`render()` draws every frame in four passes. The first two both call `drawScene()`; a `shadowPass` flag makes `draw()` send only the light's matrix in the first one. The scene pass writes three images at once (color, normals and depth) into multisampled buffers. The resolve step copies them into ordinary textures, and the outline pass reads those textures to draw the final picture.



The passes in render(), with the objects that hold each result.

Two-tone diffuse light

In class, Lambert diffuse is `Kd * max(N·L, 0)`: brightness follows the angle between the surface normal `N` and the light direction `L`, so a round object fades smoothly from light to dark. A cel painter uses two colors instead, a lit color and a shadow color, with a sharp line between them.

`calculateLight` does that by passing `N·L` through `smoothstep(0.16, 0.22, ·)` instead of `max(·, 0)`. Below 0.16 the light adds nothing; above 0.22 it adds its full amount; the 0.06 in between is a narrow ramp. The rest of the function is the Blinn-Phong code from class.



The ball, enlarged. Left: smooth Blinn-Phong. Right: our shader, with one lit tone and one shadow tone.

render.js, FRAGMENT_SHADER

```
vec3 calculateLight(int i, vec3 N, vec3 V, vec3 color) {
  vec3 L = normalize(uLightDir[i]);
  vec3 H = normalize(L + V);
  float diff = smoothstep(0.16, 0.22, dot(N, L));
  float spec = smoothstep(0.45, 0.50, pow(max(dot(N, H), 0.0), uShininess)) * diff;
  return uLightColor[i] * (Kd * diff * color + uKs * spec);
}

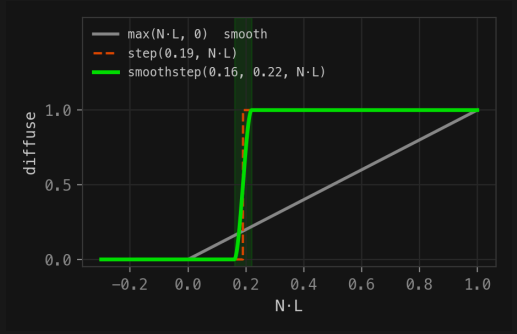
void main() {
  vec3 color = uMapping == NO_MAPPING ? uColor : uColor * texture(uTexture, vTexCoord).rgb;
  vec3 N = normalize(vNormal);
  vec3 V = normalize(-vPosition);
  float shadow = smoothstep(0.3, 0.7, texture(uShadowMap, vShadowCoord));

  vec3 result = Ka * color + uEmissive;
  result += shadow * calculateLight(0, N, V, color);
  result += calculateLight(1, N, V, color);
  fragColor = vec4(result, 1.0);
  fragNormal = vec4(N * 0.5 + 0.5, 1.0);
}
```

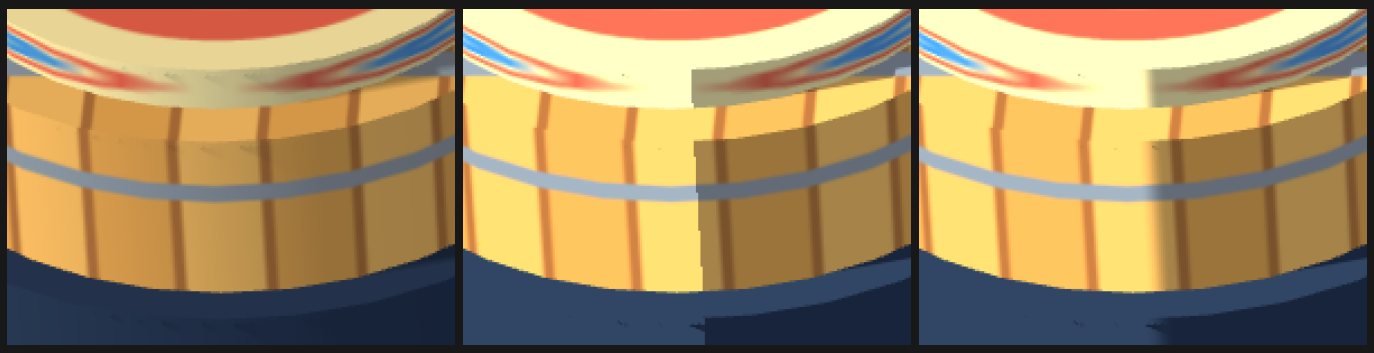
Why smoothstep and not step?

A plain `step(0.19, N·L)` also gives two tones, but its edge is exactly one pixel wide. As the ball rolls, the flippers swing and the board tilts, that edge snaps from pixel to pixel and crawls. The 0.06-wide ramp spreads the edge over a pixel or two, which is antialiasing done inside the shader.

The band sits near 0.19 rather than at 0, so surfaces that the light only grazes already count as shadow. That keeps the lit side one clean, even color.



Diffuse light as a function of N·L: smooth Lambert (gray), a hard step (orange), and our smoothstep (green).



The side of a bumper, enlarged. Left: smooth. Middle: step(0.19, N·L), a hard one-pixel edge. Right: smoothstep(0.16, 0.22, N·L), the same hard look with a slightly softened edge.

A shadow side that isn't black

The ambient term is `Ka * color`, and `Ka` is a cool gray-blue rather than a single number. The shadow tone of every surface is a darker, bluer version of its own color, the way a painter shades with cool colors instead of black. `uEmissive` lets the bumpers flash when they are hit.

render.js, FRAGMENT_SHADER

```
const vec3 Ka = vec3(0.46, 0.50, 0.58);
const float Kd = 0.6;
```

Why the lights are directional

`LIGHTS` stores a position for the key light and the fill light, but `render()` turns each one into a direction, from the table center toward the light, in eye space. A flat surface then has a single `N·L` everywhere, so the whole playfield is one flat tone.

render.js

```
const LIGHTS = [
  { pos: [-22, 8, 22], color: [1.05, 1.0, 0.9] },
  { pos: [10, -8, 14], color: [0.2, 0.25, 0.32] },
];
```

render.js, render()

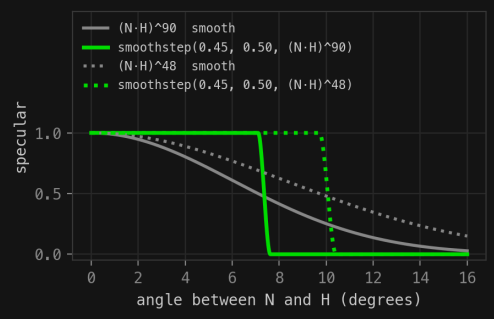
```
gl.uniform3fv(uniforms.uLightDir, LIGHTS.flatMap(l => transformDirection(view, l.pos.map((v, i) => v - TABLE_CENTER[i]))));
```

> What went wrong first: we started with point lights, where `L` changes from point to point across the floor. With a hard threshold, the line where `N·L` crossed `0.19` painted a big pale arc across the playfield. Directional lights removed it.

A highlight that is a dot

Blinn-Phong's highlight is `(N·H)^shininess`, where `H` is the vector halfway between the light and view directions. It is brightest where `N` points at `H` and fades out around that spot.

We threshold it the same way: `smoothstep(0.45, 0.50, …)` turns the fade into a solid spot with a crisp edge. Shininess still sets the spot's size, because a higher exponent makes the curve fall faster. The ball (shininess 90) gets a smaller dot than the bumper caps (48). Multiplying by `diff` keeps highlights off the shadow side.



With shininess 90 the spot ends about 7.4° from H; with 48, at about 10°.

Shadows with a hard edge

Shadows use the shadow mapping from class. First, a depth-only pass draws the table from the key light into a 2048 × 2048 depth texture, `shadowMap`. The light looks at the table through an orthographic box that `lightProjection()` fits around the cabinet corners of every field, so the whole table fits in the map with few wasted texels.

render.js

```
const lightView = lookAt(LIGHTS[0].pos, TABLE_CENTER, [0, 0, 1]);

function lightProjection() {
  const lo = [Infinity, Infinity, Infinity], hi = [-Infinity, -Infinity, -Infinity];
  for (const [x, y, z] of table.fields.flatMap(f => fitCorners(rect(f))))
    for (const dy of [0, -1.5]) {
      const p = transformPoint(lightView, [x, y + dy, z]);
      for (let i = 0; i < 3; i++) { lo[i] = Math.min(lo[i], p[i]); hi[i] = Math.max(hi[i], p[i]); }
    }
  return ortho(lo[0], hi[0], lo[1], hi[1], -hi[2] - 1, -lo[2] + 1);
}

const lightViewProjection = multiply(lightProjection(), lightView);
const shadowMatrix = multiply(translate(0.5, 0.5, 0.5), scale(0.5, 0.5, 0.5), lightViewProjection);
```

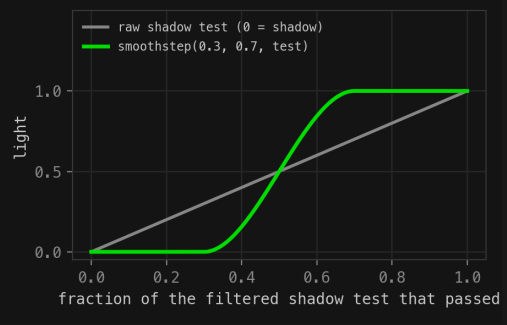
In the scene pass, the vertex shader moves each vertex into the light's texture space with `uShadowMatrix`, which is `shadowMatrix` times the model matrix. The fragment shader then asks the depth texture whether anything is closer to the light. The texture is a `sampler2DShadow` with `COMPARE_REF_TO_TEXTURE` and `LINEAR` filtering, so the GPU does the depth comparison itself and blends the results of neighboring texels.

render.js

```
const shadowMap = gl.createTexture();
gl.bindTexture(gl.TEXTURE_2D, shadowMap);
gl.texStorage2D(gl.TEXTURE_2D, 1, gl.DEPTH_COMPONENT24, SHADOW_MAP_SIZE, SHADOW_MAP_SIZE);
gl.texParameteri(gl.TEXTURE_2D, gl.TEXTURE_MIN_FILTER, gl.LINEAR);
gl.texParameteri(gl.TEXTURE_2D, gl.TEXTURE_WRAP_S, gl.CLAMP_TO_EDGE);
gl.texParameteri(gl.TEXTURE_2D, gl.TEXTURE_WRAP_T, gl.CLAMP_TO_EDGE);
gl.texParameteri(gl.TEXTURE_2D, gl.TEXTURE_COMPARE_MODE, gl.COMPARE_REF_TO_TEXTURE);
const shadowFramebuffer = gl.createFramebuffer();
gl.bindFramebuffer(gl.FRAMEBUFFER, shadowFramebuffer);
gl.framebufferTexture2D(gl.FRAMEBUFFER, gl.DEPTH_ATTACHMENT, gl.TEXTURE_2D, shadowMap, 0);
```

`texture(uShadowMap, vShadowCoord)` therefore returns a fraction, from 0 (in shadow) to 1 (lit), that changes over about one texel at a shadow's edge. `smoothstep(0.3, 0.7, …)` squeezes that back into a hard edge, so shadows match the two-tone lighting.

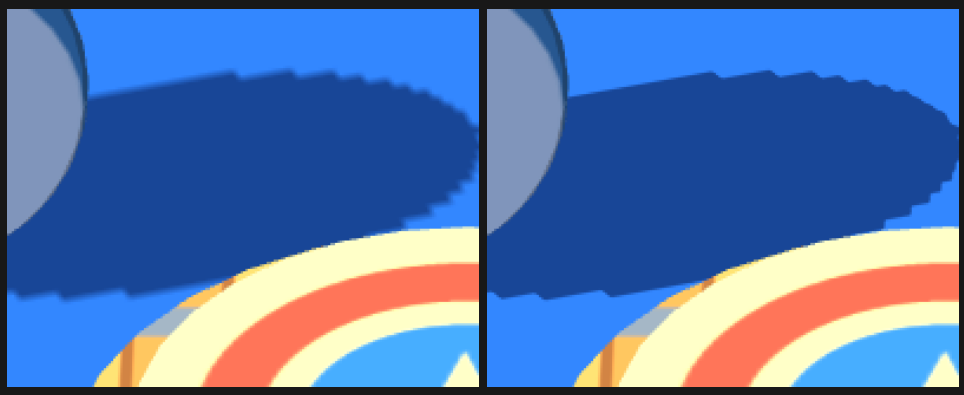
The shadow scales only the key light (light 0). The fill light and the ambient term still reach shadowed surfaces, so a shadow is a darker blue, not a black hole. During the depth pass, `polygonOffset(2, 4)` pushes the stored depths slightly away from the light, which stops surfaces from shadowing themselves ("shadow acne").



The shadow test before (gray) and after (green) the smoothstep.

render.js, render()

```
gl.useProgram(shadowProgram);
gl.bindFramebuffer(gl.FRAMEBUFFER, shadowFramebuffer);
gl.viewport(0, 0, SHADOW_MAP_SIZE, SHADOW_MAP_SIZE);
gl.clear(gl.DEPTH_BUFFER_BIT);
gl.enable(gl.POLYGON_OFFSET_FILL);
gl.polygonOffset(2, 4);
shadowPass = true;
drawScene();
shadowPass = false;
gl.disable(gl.POLYGON_OFFSET_FILL);
```



A bumper's shadow edge, enlarged. Left: the raw filtered test, a soft edge about one texel wide. Right: after smoothstep(0.3, 0.7). The stair steps are the texels of the shadow map.

Ink outlines

The outlines are not part of any model. They are found after the scene is drawn, by looking at each pixel's neighbors: wherever depth jumps or the surface direction changes sharply, the pixel is painted with `OUTLINE_COLOR`, a dark violet rather than pure black.

render.js

```
const OUTLINE_COLOR = [0.08, 0.07, 0.12];
```

Step 1: write normals next to the color

The scene shader has two outputs. Location 0 is the shaded color; location 1 is the eye-space normal, packed from [−1, 1] into [0, 1] with `N * 0.5 + 0.5` so it fits in an ordinary RGBA8 texture. The depth buffer comes for free. Together these three images are a small G-buffer.

render.js, FRAGMENT_SHADER

```
layout(location = 0) out vec4 fragColor;
layout(location = 1) out vec4 fragNormal;
```

render.js

```
const targets = [
  createTarget(gl.COLOR_ATTACHMENT0, gl.RGBA8, gl.RGBA, gl.UNSIGNED_BYTE),
  createTarget(gl.COLOR_ATTACHMENT1, gl.RGBA8, gl.RGBA, gl.UNSIGNED_BYTE),
  createTarget(gl.DEPTH_ATTACHMENT, gl.DEPTH_COMPONENT24, gl.DEPTH_COMPONENT, gl.UNSIGNED_INT),
];
const [sceneTarget, normalTarget, depthTarget] = targets;
const samples = Math.min(4, gl.getParameter(gl.MAX_SAMPLES));

const msaaFramebuffer = gl.createFramebuffer();
const resolveFramebuffer = gl.createFramebuffer();
for (const t of targets) {
  gl.bindFramebuffer(gl.FRAMEBUFFER, msaaFramebuffer);
  gl.framebufferRenderbuffer(gl.FRAMEBUFFER, t.attachment, gl.RENDERBUFFER, t.renderbuffer);
  gl.bindFramebuffer(gl.FRAMEBUFFER, resolveFramebuffer);
  gl.framebufferTexture2D(gl.FRAMEBUFFER, t.attachment, gl.TEXTURE_2D, t.texture, 0);
}
gl.bindFramebuffer(gl.FRAMEBUFFER, msaaFramebuffer);
gl.drawBuffers([gl.COLOR_ATTACHMENT0, gl.COLOR_ATTACHMENT1]);
```

Step 2: resolve the multisampled buffers

The scene pass draws into 4× multisampled renderbuffers, which smooths the edges of the geometry. But a GLSL ES 3.00 shader can't read a multisampled buffer, so `resolveTargets()` uses `blitFramebuffer` to average the samples down into the three textures that the outline pass reads. A blit writes to every draw buffer that is enabled, so it runs twice: once for color 0 together with depth, and once for color 1 (the normals) on its own.

render.js

```
function resolveTargets() {
  const w = canvas.width, h = canvas.height;
  gl.bindFramebuffer(gl.READ_FRAMEBUFFER, msaaFramebuffer);
  gl.bindFramebuffer(gl.DRAW_FRAMEBUFFER, resolveFramebuffer);
  gl.readBuffer(gl.COLOR_ATTACHMENT0);
  gl.drawBuffers([gl.COLOR_ATTACHMENT0, gl.NONE]);
  gl.blitFramebuffer(0, 0, w, h, 0, 0, w, h, gl.COLOR_BUFFER_BIT | gl.DEPTH_BUFFER_BIT, gl.NEAREST);
  gl.readBuffer(gl.COLOR_ATTACHMENT1);
  gl.drawBuffers([gl.NONE, gl.COLOR_ATTACHMENT1]);
  gl.blitFramebuffer(0, 0, w, h, 0, 0, w, h, gl.COLOR_BUFFER_BIT, gl.NEAREST);
}
```

Step 3: one triangle that covers the screen

The outline pass has no vertex buffer. The vertex shader picks one of three corners with `gl_VertexID`, making a triangle big enough to cover the whole screen, and `drawArrays(gl.TRIANGLES, 0, 3)` runs the fragment shader once per pixel.

render.js, POST_VERTEX_SHADER

```
const vec2 corners[3] = vec2[3](vec2(-1.0, -1.0), vec2(3.0, -1.0), vec2(-1.0, 3.0));
out vec2 vUv;

void main() {
  vec2 p = corners[gl_VertexID];
  vUv = p * 0.5 + 0.5;
  gl_Position = vec4(p, 0.0, 1.0);
}
```

Step 4: find the edges

render.js, POST_FRAGMENT_SHADER

```
float linearDepth(vec2 uv) {
  float z = texture(uDepth, uv).r * 2.0 - 1.0;
  return uDepthParams.y / (z + uDepthParams.x);
}

float normalDiff(vec3 n, vec2 uv) {
  return 1.0 - dot(n, texture(uNormals, uv).rgb * 2.0 - 1.0);
}

void main() {
  vec2 dx = vec2(uOutlineWidth / float(textureSize(uDepth, 0).x), 0.0);
  vec2 dy = vec2(0.0, uOutlineWidth / float(textureSize(uDepth, 0).y));
  float d = linearDepth(vUv);
  float depthEdge = (abs(linearDepth(vUv - dx) + linearDepth(vUv + dx) - 2.0 * d)
                  + abs(linearDepth(vUv - dy) + linearDepth(vUv + dy) - 2.0 * d)) / d;
  vec3 n = texture(uNormals, vUv).rgb * 2.0 - 1.0;
  float normalEdge = max(max(normalDiff(n, vUv - dx), normalDiff(n, vUv + dx)),
                         max(normalDiff(n, vUv - dy), normalDiff(n, vUv + dy)));
  float edge = max(smoothstep(0.02, 0.05, depthEdge), smoothstep(0.3, 0.5, normalEdge));
  fragColor = vec4(mix(texture(uScene, vUv).rgb, uOutlineColor, edge), 1.0);
}
```

Linear depth. The depth texture stores `(z_ndc + 1) / 2`, which is not proportional to distance. Our projection matrix (the handout's `perspective()`, or `frustum()` for the off-center table camera) gives `z_clip = P[10] * z_eye + P[14]` and `w_clip = -z_eye`. Dividing, `z_ndc = -P[10] - P[14] / z_eye`, and solving for the distance:

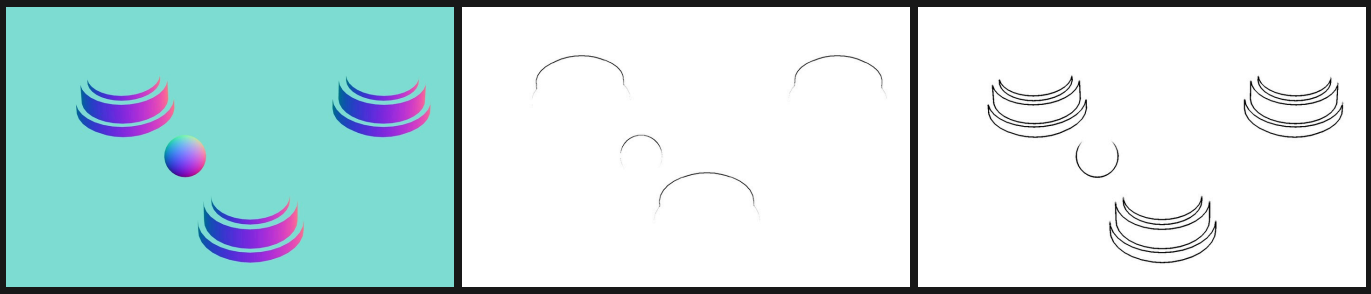
```
distance = -z_eye = P[14] / (z_ndc + P[10])
```

That is `linearDepth()`, with `uDepthParams = (P[10], P[14])` passed in by `render()`.

Depth edges. `depthEdge` adds the second difference of that distance along x and along y, for example `d(left) + d(right) - 2 * d(center)`, and divides by `d`. On a flat surface depth changes at an almost steady rate from pixel to pixel, so the second difference is close to zero, even on the playfield seen at a low angle. At a silhouette, where a bumper ends and the floor behind it begins, the distance jumps and the second difference is large. Dividing by `d` makes the test relative, so one threshold works near the camera and far away. A first difference would not work: the playfield's depth grows steadily toward the top of the screen, so every pixel of it would count as an edge.

Normal edges. `normalDiff` is `1 - n·n'` for a neighbor's normal `n'`: 0 where neighbors face the same way, 1 at a right angle. `smoothstep(0.3, 0.5)` starts drawing at about 46° and is solid by 60°. That catches creases that depth misses, such as the rings around the bumpers and the corners where wall tops meet wall sides. The 11.25° steps between the 32 faces of a cylinder stay unlined, so round things still look round.

Combining. The two tests are joined with `max`, and `mix` moves the scene color toward the outline color by that amount.



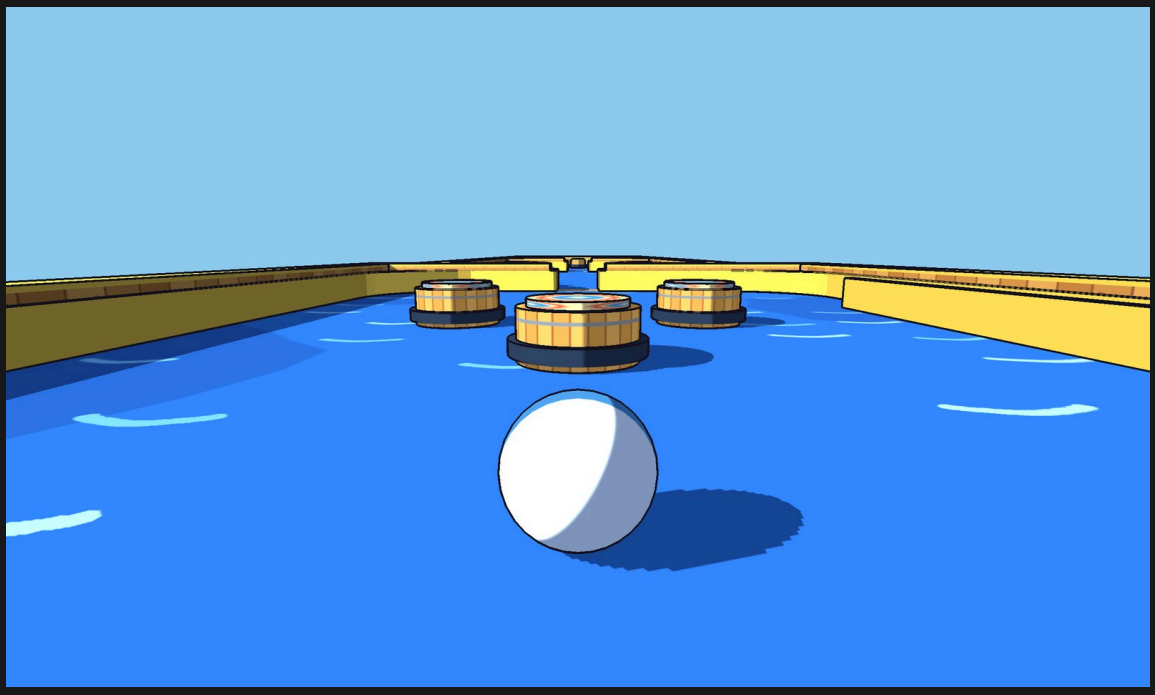
What the outline pass sees, for the same frame as above. Left: the normal buffer (x, y, z as red, green, blue). Middle: depth edges only, the silhouettes. Right: normal edges only, the creases. The final outline is the darker of the two at each pixel.

Step 5: run it

After the resolve, `render()` switches to the default framebuffer (the canvas), binds the three textures, and draws the triangle. `uOutlineWidth` is the device pixel ratio: on a Retina screen it is 2, so neighbors are sampled 2 pixels away and the lines come out 2 pixels thick, the same size on screen as on a normal display.

render.js, render()

```
  gl.bindFramebuffer(gl.FRAMEBUFFER, null);
  gl.useProgram(postProgram);
  gl.uniform2f(postUniforms.uDepthParams, projection[10], projection[14]);
  gl.uniform1f(postUniforms.uOutlineWidth, outlineWidth);
  gl.activeTexture(gl.TEXTURE2);
  gl.bindTexture(gl.TEXTURE_2D, depthTarget.texture);
  gl.activeTexture(gl.TEXTURE1);
  gl.bindTexture(gl.TEXTURE_2D, normalTarget.texture);
  gl.activeTexture(gl.TEXTURE0);
  gl.bindTexture(gl.TEXTURE_2D, sceneTarget.texture);
  gl.disable(gl.DEPTH_TEST);
  gl.bindVertexArray(null);
  gl.drawArrays(gl.TRIANGLES, 0, 3);
  gl.enable(gl.DEPTH_TEST);
}
```

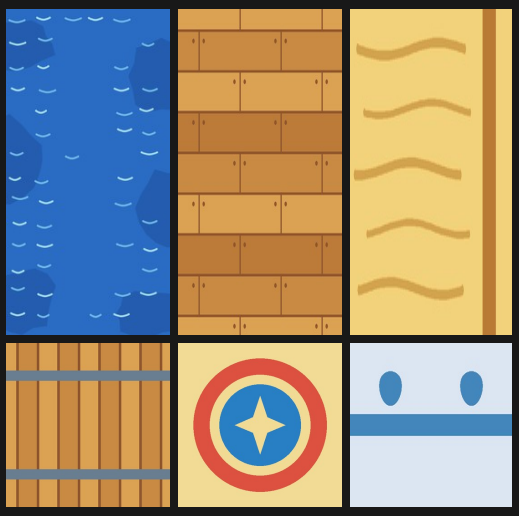


Ball cam (press C). The same shaders in a low perspective view: two-tone bumpers, the ball's lit and shadow halves, hard shadows, and outlines on every silhouette.

Textures

Every material uses a flat, painted-looking texture: the sea for the playfield, planks for the cabinet, wood for the flippers, and painted bumpers, caps and ball. None of them has a normal map. Small bumps from a normal map would break the flat bands up into noise; in cel shading the detail comes from the color texture, the two-tone light and the outlines.

Each material lists its own `ks` and `shininess` for the highlight, and an optional `repeat` that tiles a texture across a stretched cube face, for example `[1, 6]` down the long side walls.



sea, planks, wood / bumper, cap, ball (textures/toon/)

render.js

```
const MATERIALS = {
  playfield: { ks: 0.25, shininess: 20, texture: seaTexture },
  cabinet: { ks: 0.1, shininess: 24, texture: planksTexture, repeat: [4, 1] },
  cabinetSide: { ks: 0.1, shininess: 24, texture: planksTexture, repeat: [1, 6] },
  rail: { color: [0.98, 0.80, 0.30], ks: 0.5, shininess: 64 },
  bumper: { ks: 0.15, shininess: 32, texture: bumperTexture },
  bumperCap: { ks: 0.3, shininess: 48, texture: capTexture },
  flipper: { ks: 0.25, shininess: 40, texture: woodTexture },
  rubber: { color: [0.16, 0.22, 0.30], ks: 0.15, shininess: 24 },
  ball: { ks: 0.6, shininess: 90, texture: ballTexture },
  steel: { color: [0.86, 0.90, 0.95], ks: 0.6, shininess: 90 },
};
```

What it costs

The extra work over plain Blinn-Phong is the second color attachment and one full-screen pass with 11 texture reads per pixel (the scene color, 5 depths and 5 normals). On our 2019 MacBook Pro, the integrated Intel GPU managed about 14 fps at Retina resolution with 4× MSAA. Asking the browser for the discrete AMD GPU with `getContext("webgl2", { powerPreference: "high-performance" })` brings it to 60 fps. The shadow pass cost nothing we could measure.

One thing we would still improve: the edges are found after the MSAA resolve, so the outlines themselves are not antialiased and keep a little stair-stepping. An FXAA pass after the outline pass would smooth them.